

Python Dictionaries: 25 Practice Problems

Comprehensive University & AI/ML Coursework Preparation Set

Topic: Dictionaries

Progression: Basic to Advanced

16+ Loop Problems

12+ Nested Structures

PART 1: BASIC OPERATIONS & FUNDAMENTALS (Q1 – Q5)

Q01 The Bookstore Inventory

Create a dictionary representing a bookstore's inventory where keys are book titles and values are quantities in stock. Perform the following operations: add a new book to the inventory, update the stock of an existing book by adding 15 copies, and print the total number of unique book titles currently in the system.

Concepts Tested: Dictionary creation, adding/updating key-value pairs, `len()`.

Q02 User Authentication Check

You have a dictionary of registered usernames mapping to their account statuses ("Active", "Banned", or "Suspended"). Write a script that takes a specific username as input. If the username exists in the dictionary, print their status. If the username does not exist, add them to the dictionary with a default status of "Active".

Concepts Tested: Checking if a key exists, accessing values, conditional statements with dictionaries.

Q03 Support Ticket Cleanup

Start with a dictionary of active IT support tickets where keys are ticket IDs and values are issue descriptions. Perform exactly four operations:

1. Use `del` to permanently remove a specific ticket ID (e.g., a spam ticket).
2. Use `pop()` to remove and process a high-priority ticket, storing its description in a variable.
3. Use `popitem()` to remove the most recently added ticket.
4. Use `clear()` to close all remaining tickets at the end of the day.

Concepts Tested: Deletion methods (`del`, `pop()`, `popitem()`, `clear()`).

Q04 Movie Rating Formatter

Given a dictionary mapping movie titles to their IMDB ratings, use a `for` loop to iterate through the data and print each entry in a clean format.

```
Example Input: {"Inception": 8.8, "The Matrix": 8.7, "Interstellar": 8.6}
Expected Output:
Movie: Inception - Rating: 8.8/10
Movie: The Matrix - Rating: 8.7/10
Movie: Interstellar - Rating: 8.6/10
```

Concepts Tested: Iterating through dictionaries, `items()`, looping through keys and values.

Q05 Cricket Player Roster

You are given two separate lists: one containing the names of five cricket players, and another containing their respective match scores. Write a script using a `for` loop (or `zip()`) to convert these two lists into a single dictionary mapping player names to their scores.

Concepts Tested: Converting lists/data into dictionaries, dynamic dictionary creation.

PART 2: INTERMEDIATE DATA MANIPULATION & LOOPS (Q6 – Q11)

Q06 Word Frequency Counter

Given a string representing a paragraph of text, count the frequency of each word and store the result in a dictionary where the keys are words and the values are their occurrence counts. (Assume punctuation is already removed).

Concepts Tested: Counting/frequency problems, loops with conditions, dictionary updating.

Q07 Highest and Lowest Earner

Given a dictionary of company employees and their annual salaries, find and print the name of the employee with the highest salary and the one with the lowest salary.

Constraint: You must use a `for` loop to find these values, not the built-in `max()` or `min()` functions directly on the dictionary values.

Concepts Tested: Finding maximum/minimum values, looping through items, relational operators.

Q08 Grouping Students by Grade

You are given a list of dictionaries, where each dictionary represents a student with keys `'name'` and `'grade'` (e.g., `'A'`, `'B'`, `'C'`). Create a new dictionary where the keys are the distinct grades, and the values are lists of student names who received that grade.

```
Example Input: [{"name": "Ali", "grade": "A"}, {"name": "Sara", "grade": "B"}, {"name": "Zain", "grade": "A"}]
Expected Output: {"A": ["Ali", "Zain"], "B": ["Sara"]}
```

Concepts Tested: Handling duplicate or repeated data, list of dictionaries, appending to lists inside dictionaries.

Q09 Climate Data Conversion

Given a dictionary mapping international cities to their current temperatures in Celsius, use a **dictionary comprehension** to create a new dictionary where the temperatures are converted to Fahrenheit. Formula: $(C * 9/5) + 32$.

Concepts Tested: Dictionary comprehensions, data transformation.

Q10 Merging Branch Expenses

You have two dictionaries representing monthly expenses for two different branches of a retail company. The keys are expense categories (e.g., "Rent", "Supplies", "Marketing") and values are the costs. Merge them into a single dictionary. If a category exists in both branches, sum their expenses together in the final dictionary.

Concepts Tested: Working with multiple dictionaries, merging dictionaries, conditional logic.

Q11 Bank Account Logic

Given a dictionary of bank account numbers (keys) and their current balances (values), loop through the dictionary to apply banking rules. If the balance is below \$100, deduct a \$10 maintenance fee. If the balance is strictly over \$1,000, add a 2% interest bonus. Update the original dictionary with the new balances.

Concepts Tested: Updating records, using loops with conditions, modifying dictionaries in place.

PART 3: NESTED STRUCTURES & TRAVERSAL (Q12 – Q20)

Q12 E-commerce Price Filter (Nested)

Given a nested dictionary of e-commerce inventory where the top-level keys are categories, and values are dictionaries mapping product names to prices, write a script with nested loops to extract all products priced below \$50 into a new, single-level dictionary.

```
Example Input: {"Electronics": {"Mouse": 25, "Monitor": 150}, "Clothing": {"Shirt": 30, "Jacket": 80}}
Expected Output: {"Mouse": 25, "Shirt": 30}
```

Concepts Tested: Nested dictionaries, filtering, nested loops.

Q13 API Data Validation

You have an `expected_schema` dictionary mapping field names to their expected Python data types (e.g., `{"username": str, "age": int}`). You receive an `api_response` dictionary. Write a loop to validate the response. The script should verify that all keys in the schema exist in the response, and that the value types match. Print a list of any missing keys or type mismatches.

Concepts Tested: Comparing dictionaries, data validation, `keys()`, `type()`.

Q14 Leaderboard Ranking

Given a dictionary mapping online gamers' usernames to their total score, create a script that sorts the dictionary by score in descending order, printing a formatted 1st to Nth place leaderboard.

Concepts Tested: Sorting dictionary data by values, ranking.

Q15 HR Record Update (Nested)

Given a nested dictionary mapping Employee IDs to their profile (`{"name": "...", "department": "...", "salary": ...}`), write a script that searches for a specific employee by their *name* (not ID). Once found, update their department to "Data Science" and increase their salary by 10%.

Concepts Tested: Nested dictionary traversal (Dictionary → dictionary), searching, updating records.

Q16 Student GPA Calculator (Nested)

You are given a nested dictionary where keys are student names, and values are dictionaries of subject-to-marks mappings. Write a script to calculate the average mark for each student, and output a new dictionary mapping student names to their calculated average.

```
Example Input: {"Ayesha": {"Math": 90, "Physics": 85}, "Bilal": {"Math": 70, "Physics": 75}}
Expected Output: {"Ayesha": 87.5, "Bilal": 72.5}
```

Concepts Tested: Nested loops, calculating averages, building dictionaries inside loops.

Q17 Corporate Deep Search (Nested)

Given a company structure dictionary mapping Divisions → Departments → Lists of employee names, write a script to search for a specific employee's name. If found, print a string formatted as "Found in: [Division] - [Department]".

Concepts Tested: Dictionary → dictionary → list nesting, deep searching, multi-level iteration.

Q18 Global Population Aggregation (Nested)

Given a dictionary containing demographic data formatted as Country -> City -> Population, calculate two things: a new dictionary mapping each country to its total national population, and an integer representing the overall global population across the entire dataset.

Concepts Tested: Nested dictionaries, counting and aggregating data using loops.

Q19 Flattening a JSON Payload (Nested)

Given a complex nested dictionary simulating an API JSON response, flatten it into a single-level dictionary. Combine nested keys using dot notation.

```
Example Input: {"user": {"id": 101, "details": {"name": "Tariq", "city": "Lahore"}},
               "status": "active"}
Expected Output: {"user.id": 101, "user.details.name": "Tariq", "user.details.city":
               "Lahore", "status": "active"}
```

Concepts Tested: Data transformation, recursive logic or nested loop state tracking, dictionary flattening.

Q20 Inverting University Enrollments (Nested)

Given a nested dictionary of university courses mapped to enrolled student IDs and their grades (e.g., `Course Code -> Student ID -> Grade`), invert the structure. Create a new nested dictionary organized by students (e.g., `Student ID -> Course Code -> Grade`).

Concepts Tested: Creating new nested dictionaries from existing data, structural inversion, nested loops.

PART 4: ADVANCED LOGIC & AI/ML WORKFLOWS (Q21 – Q25)

Q21 Senior Faculty Filter (Nested Comprehension)

Given a nested dictionary of university departments and their staff (`Department -> Professor Name -> Years of Experience`), use a **nested dictionary comprehension** to create a new structure keeping only professors with more than 15 years of experience. If a department is left with zero professors after filtering, it should not appear in the final dictionary.

Concepts Tested: Nested dictionary comprehensions, conditional filtering, removing empty structures.

Q22 Deep Config Merge (Nested)

You have two nested configuration dictionaries (e.g., `default_settings` and `user_settings`). Write a script to perform a "deep merge". If a key exists in both and points to a dictionary, merge those inner dictionaries recursively. If a key points to a standard value (int, string), `user_settings` should overwrite `default_settings`.

Concepts Tested: Deep merging, handling overlapping keys in nested structures, dynamic type checking.

Q23 MVP Finder Across Teams (Nested)

Given a dictionary mapping sports teams to dictionaries of players and their total seasonal points, find the single player with the highest overall points across *all* teams combined. Print their name, their team, and their score.

Concepts Tested: Finding highest/lowest values across multiple dictionary depths, state tracking in loops.

Q24 Transaction Parser (Dynamic Nesting)

You are given a raw list of tuples representing sales transactions: `[(Year, Month, Region, Revenue), ...]`. Dynamically build a nested dictionary structure from this list: `Year -> Month -> Region -> Total Revenue`. If multiple tuples have the exact same Year, Month, and Region, their revenues must be aggregated (summed) in the dictionary.

Concepts Tested: Dynamic nested dictionary creation, list-to-nested-dictionary conversion, aggregation.

Q25 The Final Challenge: ML Evaluation Matrix (Nested)

You are given a complex dataset represented as a dictionary `results` mapping Model Names (e.g., "RandomForest", "NeuralNet") → Dataset Names (e.g., "Dataset_A", "Dataset_B") → dictionary of metrics (`{"accuracy": float, "f1_score": float, "latency_ms": int}`).

Write a comprehensive script to do the following:

1. Calculate the "macro-average" accuracy and `f1_score` across all datasets for each model.
2. Normalize the structure: If any dataset is missing the "latency_ms" key, infer it by calculating the average latency of that specific model across its *other* datasets and insert it.
3. Identify which model has the highest macro-average `f1_score`.
4. Output a summary nested dictionary mapping each model to its aggregated stats: `{"modelName": {"macro_accuracy": x, "macro_f1": y, "avg_latency": z}}`.

Concepts Tested: Real-world AI data structures, 3-level deep nested dictionaries, dynamic missing key handling, multi-metric aggregation, data normalization, ranking.